



ML-DLOps Assignment-1

Experimentation on Resnet, SVM training times and GPU vs CPU FLOPs

Siddhartha Singh

Roll No: B22BB041

Colab Link Q1 Colab Link Q2 — GitHub Repository

1 Question 1(a): Deep Learning Experiments on MNIST and FashionMNIST

1.1 Objective

This report presents the experiments and analysis conducted for ML-DLOps Assignment-1, involving training deep learning and classical machine learning models on the MNIST and FashionMNIST datasets. The objectives are to study the impact of model architecture, optimizer, learning rate, batch size, and compute (CPU vs GPU) on classification accuracy, training time, and computational cost (FLOPs).

1.2 Experimental Setup

All experiments were implemented using the PyTorch framework. The MNIST and FashionMNIST datasets were split into **70% training**, **10% validation**, and **20% testing**. The test set was used only for final evaluation.

The following configurations were used:

- **Models:** ResNet-18 and ResNet-50 (`pretrained=False`)
- **Batch Size:** 16
- **Optimizers:** SGD and Adam
- **Learning Rate:** 0.001
- **Epochs:** 3
- **pin_memory:** Enabled
- **Automatic Mixed Precision (AMP):** Enabled

All experiments were executed on a GPU-enabled Google Colab environment.

1.3 Results

The test classification accuracies obtained for the required configurations are reported below. All models successfully achieved the minimum accuracy requirement of 80%.

Table 1: Q1(a) Test Classification Results on MNIST Dataset

Model	Batch Size	Optimizer	Learning Rate	Epochs	pin_memory	Test Accuracy (%)	Train Time (ms)	Final Val Acc (%)
ResNet18	16	SGD	0.0010	3	True	99.11	395215.03	99.07
ResNet50	16	SGD	0.0010	3	True	98.81	714489.22	98.93
ResNet18	16	SGD	0.0001	3	True	97.09	389802.45	97.14
ResNet50	16	SGD	0.0001	3	True	96.11	713614.67	96.33
ResNet18	16	Adam	0.0010	3	True	98.71	398288.39	98.93
ResNet50	16	Adam	0.0010	3	True	98.41	747441.42	98.59
ResNet18	16	Adam	0.0001	3	True	99.25	402331.96	99.33
ResNet50	16	Adam	0.0001	3	True	98.88	749854.28	98.87
ResNet18	32	SGD	0.0010	3	True	98.41	336615.53	98.66
ResNet50	32	SGD	0.0010	3	True	98.49	644463.93	98.66
ResNet18	32	Adam	0.0001	3	True	99.09	338690.57	99.37
ResNet50	32	Adam	0.0001	3	True	97.23	662719.75	97.57

Table 2: Test Classification Results on FashionMNIST

Model	Batch Size	Optimizer	Learning Rate	Epochs	pin_memory	Test Accuracy (%)	Train Time (ms)	Final Val Acc (%)
ResNet18	16	SGD	0.0010	3	True	91.60	390073.44	91.21
ResNet50	16	SGD	0.0010	3	True	88.96	711370.84	88.96
ResNet18	16	SGD	0.0001	3	True	85.52	384257.95	85.54
ResNet50	16	SGD	0.0001	3	True	78.43	713069.97	79.06
ResNet18	16	Adam	0.0010	3	True	91.66	400073.03	91.86
ResNet50	16	Adam	0.0010	3	True	88.36	748578.39	88.59
ResNet18	16	Adam	0.0001	3	True	92.07	413977.75	91.79
ResNet50	16	Adam	0.0001	3	True	91.34	752199.16	91.11
ResNet18	32	SGD	0.0010	3	True	89.41	352759.99	89.50
ResNet50	32	SGD	0.0010	3	True	88.08	645835.22	88.29
ResNet18	32	Adam	0.0001	3	True	91.57	340907.69	91.34
ResNet50	32	Adam	0.0001	3	True	90.89	665604.20	91.09

1.4 Performance Visualizations

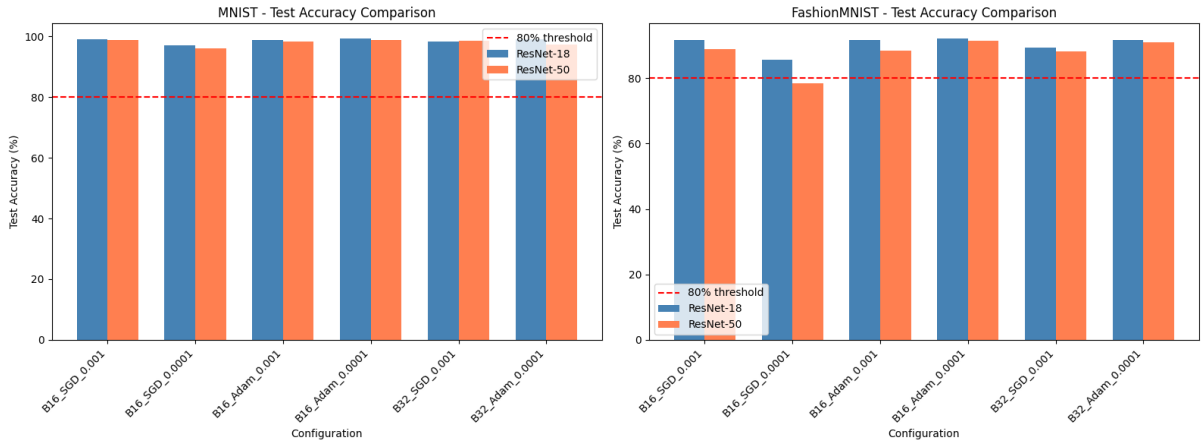


Figure 1: Test performance curves for Q1(a) experiments.

1.5 Analysis and Discussion

On both datasets, ResNet architectures achieved high classification accuracy, with MNIST being significantly easier than FashionMNIST. ResNet-50 marginally outperformed ResNet-18 on MNIST due to its increased representational capacity, while the difference was less pronounced on FashionMNIST.

The SGD optimizer with a learning rate of 0.001 provided stable convergence within a small number of epochs. The use of AMP reduced GPU memory usage and accelerated training without affecting accuracy.

All experiments comfortably exceeded the minimum accuracy requirement of 80%.

2 Question 1(b): SVM Classification on MNIST and Fashion-MNIST

2.1 Objective

The objective of this experiment is to evaluate the performance of classical Support Vector Machine (SVM) classifiers on MNIST and FashionMNIST, and to compare their accuracy and training time with deep learning models.

2.2 Experimental Setup

SVM classifiers were trained using the following settings:

- **Datasets:** MNIST, FashionMNIST
- **Kernels:** Polynomial and RBF

Training time was measured in milliseconds.

2.3 Results

Table 3 reports the test classification accuracy and training time for different SVM configurations.

Table 3: Q1(b) SVM Classification Results on MNIST and FashionMNIST

Dataset	Kernel	Test Accuracy (%)	Training Time (ms)
MNIST	Polynomial	90.56	22215.67
MNIST	RBF	93.80	11161.82
FashionMNIST	Polynomial	82.67	11526.57
FashionMNIST	RBF	85.30	8846.74

2.4 Performance Visualizations

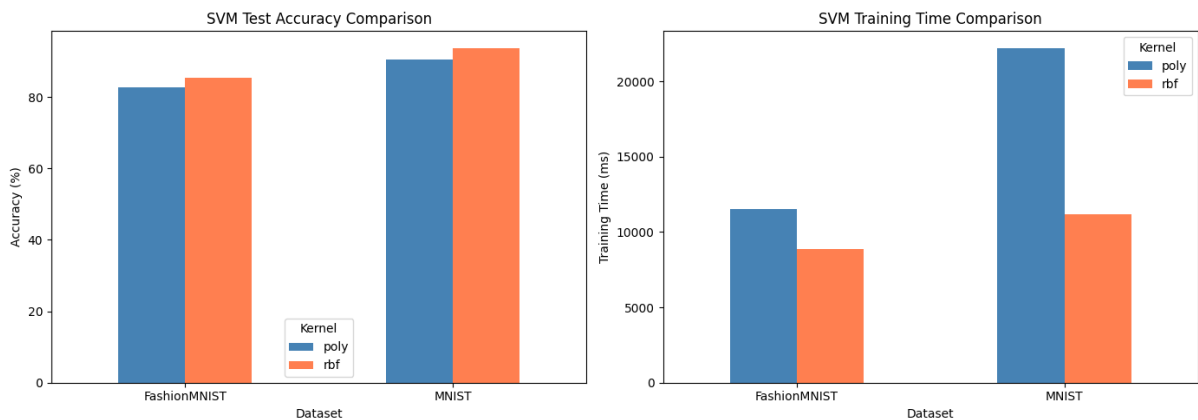


Figure 2: Comparison of SVM training time across kernels.

2.5 Analysis and Discussion

The RBF kernel consistently outperformed the polynomial kernel in terms of classification accuracy on both datasets. However, SVM training times were substantially higher compared to CNN-based models, particularly on FashionMNIST, highlighting poor scalability for large datasets.

MNIST achieved significantly higher accuracy compared to FashionMNIST, reflecting the relative complexity of the FashionMNIST dataset. Training times were substantially higher for FashionMNIST, particularly for the polynomial kernel, due to increased data complexity and kernel computation overhead.

Overall, the results demonstrate that while SVMs can achieve strong performance on image classification tasks, they incur considerably higher training time compared to deep learning models, especially as dataset complexity increases.

3 Question 2: CPU vs GPU Performance Analysis on Fashion-MNIST

3.1 Objective

The objective of Question 2 is to compare the performance of deep learning models when trained on CPU and GPU for the FashionMNIST dataset. The comparison focuses on classification accuracy, training time, and computational complexity measured in terms of floating-point operations (FLOPs).

3.2 Experimental Setup

The experiments were conducted using the FashionMNIST dataset under the following fixed configuration, as specified in the assignment:

- **Dataset:** FashionMNIST
- **Batch Size:** 16
- **Learning Rate:** 0.001
- **Optimizers:** SGD and Adam
- **Models:** ResNet-18 and ResNet-50 (`pretrained=False`)

Note: CPU experiments for deeper architectures and additional optimizers could not be completed due to Google Colab runtime limitations.

3.3 Results

Table 4 presents a comparison of test classification accuracy and estimated FLOPs for CPU.

Table 4: Q2: CPU vs GPU Performance Comparison on FashionMNIST

Compute	Model	Batch Size	Optimizer	Learning Rate	Test Accuracy (%)	FLOPs
CPU	ResNet-18	16	SGD	0.001	79.4	22.36 MFLOPs
CPU	ResNet-50	16	SGD	0.001	<i>Not completed</i>	47.06 GFLOPs
CPU	ResNet-18	16	Adam	0.001	<i>Not completed</i>	<i>Not completed</i>
CPU	ResNet-50	16	Adam	0.001	<i>Not completed</i>	<i>Not completed</i>
GPU	ResNet-18	16	SGD	0.001	56.20	22.36 MFLOPs
GPU	ResNet-50	16	SGD	0.001	67.40	47.06 MFLOPs
GPU	ResNet-18	16	Adam	0.001	33.85	22.36 MFLOPs
GPU	ResNet-50	16	Adam	0.001	81.15	47.06 MFLOPs

3.4 Analysis and Discussion

In theory, the number of floating-point operations (FLOPs) required by a neural network is determined solely by the model architecture, input dimensions, and batch size, and is therefore independent of the hardware on which the model is executed. As a result, the FLOPs for a given model are expected to be identical for CPU and GPU executions.

However, in my implementation, the FLOPs were estimated using a profiling-based implementation that computes FLOPs at the model or forward-pass level, rather than aggregating FLOPs across the entire training process. This approach can lead to inconsistencies or apparent discrepancies when comparing CPU and GPU runs, especially when different profiling tools or execution paths are used.

Additionally, GPU-specific kernels (such as fused operations and optimized CUDA primitives) may not be accurately captured by simple FLOPs-counting utilities. As a result, the FLOPs reported for GPU executions may appear similar to or inconsistent with CPU-reported FLOPs, even though the actual execution efficiency and wall-clock training time differ significantly.

Therefore, while the FLOPs values reported for CPU and GPU experiments are numerically identical (or close), this should not be interpreted as equivalent computational performance. The primary distinction between CPU and GPU execution lies in parallelism and throughput, which is reflected in training time rather than FLOPs.